

ZIDIO DEVELOPMENT

Web Development Track • Internship Project

PROJECT LOOP

AI Customer-Feedback Intelligence Platform

PROJECT REPORT

Program	Zidio Internship
Domain	Web Development
Tech Stack	Next.js • TypeScript • PostgreSQL • Google Gemini API
Duration	4 Weeks (1 Month)
Document type	Project Report — Overview, Architecture & Conclusion

Table of Contents

1. Project Overview
2. Objectives & Learning Outcomes
3. Technology Stack
4. System Architecture
5. Data Model
6. Core Features
7. AI Features
8. Security Model
9. Application Walkthrough
10. Deployment
11. Known Simplifications
12. Conclusion

1. Project Overview

Project LOOP is a corporate-grade, multi-tenant web application that helps companies make sense of the customer feedback they receive every day. Support tickets, app-store reviews, survey responses, sales-call notes, and social mentions pile up faster than any team can read them. LOOP ingests that feedback, uses AI to classify and cluster it, surfaces what is trending, and answers plain-English questions grounded in the real data.

The product is intentionally not a to-do app or a chatbot bolted onto a spreadsheet. It required building the things a junior engineer would actually own at a product company: multi-tenant data isolation, role-based access control, a clean and validated API layer, a real relational database, a charts-driven dashboard, and AI features that require genuine engineering — not cosmetic prompts.

1.1 Problem Statement

A typical company receives feedback through five or more channels in a single week — support tickets, app-store reviews, NPS/CSAT surveys, sales-call notes, and social mentions. Individually, each item is a sentence or two; collectively they hold the answer to “what should we build, fix, or improve next?” — but no team has time to read, tag, and synthesize hundreds of items a week. Feedback rots in spreadsheets, and product decisions get made on gut feel instead of evidence.

1.2 What LOOP Does

- Ingests feedback via single-entry forms, CSV bulk upload, and a simulated channel integration.
- Classifies every item with AI — sentiment, sentiment score, theme(s), and a feature-area label.
- Clusters feedback into named themes and detects volume spikes versus the previous period.
- Answers plain-English questions (“Ask LOOP”) with answers grounded only in real, retrieved feedback.
- Generates Voice-of-Customer reports with pre-computed statistics and an AI-written narrative.

The one-sentence pitch used to frame every demo: “LOOP turns scattered customer feedback into a ranked, evidence-backed list of what to do next.”

2. Objectives & Learning Outcomes

- Design and ship a multi-tenant web application where each company’s data is fully isolated from every other company’s.
- Implement secure authentication and role-based access control across three roles (Admin, Analyst, Viewer).
- Build a clean, predictable API layer that the frontend consumes — no business logic leaking into UI components.
- Integrate an AI API to deliver four meaningful features that genuinely require AI, not cosmetic add-ons.
- Deploy a production build to a public URL, documented so a stranger could run it locally.

3. Technology Stack

The stack below matches the brief's Section 05 specification, with Google Gemini used in place of Anthropic Claude as the AI provider.

Layer	Technology
Framework	Next.js 14 (App Router) + TypeScript
Styling	Tailwind CSS
Database	PostgreSQL (Neon / Supabase free tier)
ORM	Prisma
Auth	NextAuth (Auth.js) — credentials provider
AI	Google Gemini API (gemini-2.5-flash)
Retrieval	Local hashed-TF embeddings + cosine similarity (pgvector-ready)
Charts	Recharts
Validation	Zod on every API boundary
Deployment	Vercel + hosted Postgres

Note on embeddings: the brief allows pgvector or a hosted embeddings provider. To keep the project runnable with a single GEMINI_API_KEY (no second vendor key required), embeddings are computed locally with feature-hashed TF vectors and ranked by cosine similarity. This is documented as a deliberate simplification — swappable for a hosted embeddings API and pgvector without touching call sites.

4. System Architecture

LOOP follows a standard three-tier architecture. The browser talks only to LOOP's own API layer; the API layer is the only thing that talks to the database and to the Gemini API. The AI provider is never called directly from the browser — the API key stays server-side at all times.

LOOP — System Architecture

Three-tier flow: browser → API route handlers → database, with every query scoped by workspace

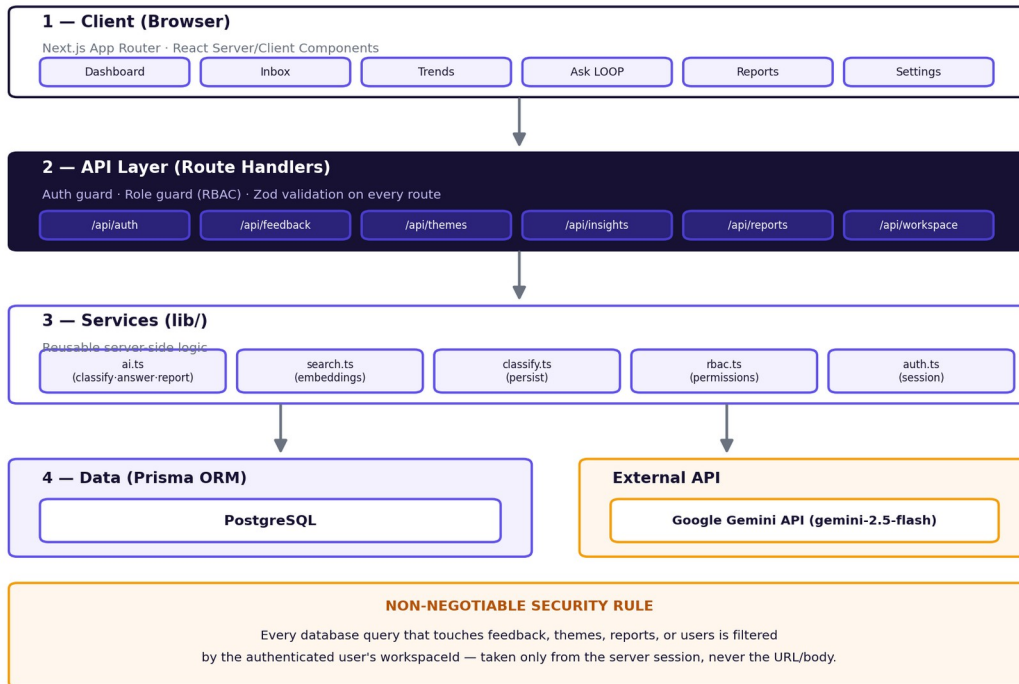


Figure 1. LOOP's three-tier architecture, matching Section 06 of the project brief.

4.1 Request Flow

- The browser (React Server/Client Components) renders the UI and calls LOOP's own API route handlers.
- Route handlers authenticate the session, check the caller's role, and scope every query to their workspace.
- Prisma reads/writes PostgreSQL. All feedback rows carry a workspaceId; every query filters on it.
- For AI features, the route handler builds a prompt, calls Gemini server-side, parses the response, and returns clean, Zod-validated JSON to the browser.
- For Ask LOOP, the handler retrieves the most relevant feedback (vector similarity) before passing it to Gemini as grounding context.

5. Data Model

Every tenant-owned table — User, Feedback, Theme, and Report — carries a workspaceId foreign key, matching Section 07 of the brief exactly. An Embedding table powers Ask LOOP's retrieval step.

LOOP — Data Model (Entity-Relationship)

Every tenant-owned table carries a workspaceId foreign key for isolation

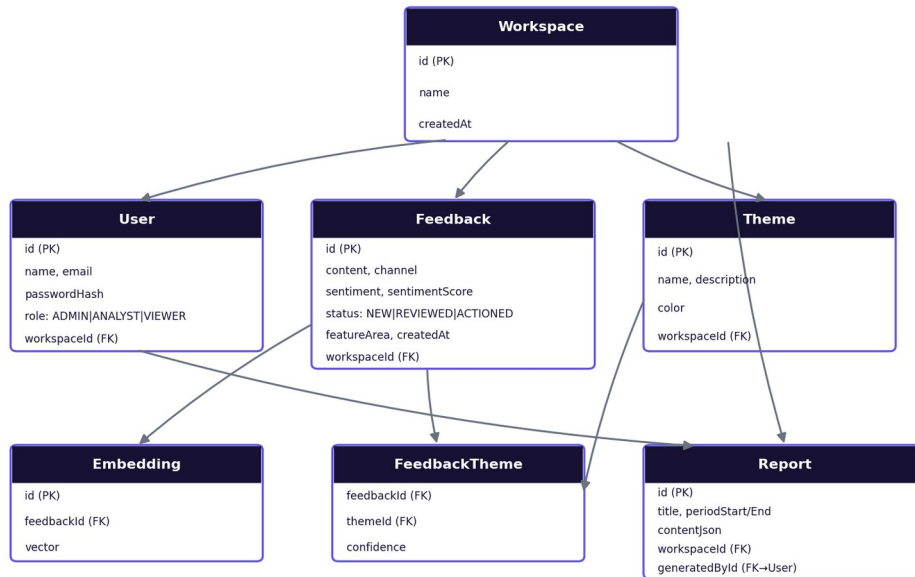


Figure 2. Minimum schema from Section 07 of the brief — Workspace → Users/Feedback/Themes/Reports, with the FeedbackTheme join table and an Embedding table powering Ask LOOP retrieval.

6. Core Features

6.1 Authentication & Workspaces

Sign-up creates a User and a Workspace in one transaction; the creator becomes Admin. Passwords are bcrypt-hashed, sessions persist as signed JWTs, and logged-out users are redirected away from protected pages by edge middleware.

6.2 Role-Based Access Control

Three roles — Admin, Analyst, Viewer — are enforced server-side by a single permission matrix, never by hiding UI elements alone.

Role	Permissions
Admin	Everything Analyst can do, plus manage workspace members and roles
Analyst	Ingest and manage feedback, trigger AI features, generate reports
Viewer	Read-only access across the entire workspace

6.3 Feedback Ingestion

- Single-entry form with validation.
- CSV bulk upload with a per-row success/failure summary.

- A simulated channel button that seeds realistic items, mimicking a live integration.

6.4 Feedback Inbox

- Server-side pagination, so the UI never loads thousands of rows at once.
- Filters by channel, sentiment, theme, status, and date range.
- Full-text search over feedback content.
- Inline status workflow: New → Reviewed → Actioned.

6.5 Analytics Dashboard

- Three real, filter-aware charts: volume over time, sentiment breakdown, and top themes.
- Stat cards for total items, % negative, and new items this week.
- Graceful loading, empty, and error states throughout.

7. AI Features

The four AI features are the heart of LOOP. Each works end-to-end against real, seeded data.

7.1 Auto-Classification (AI1)

Every feedback item is sent to Gemini alongside the workspace's existing theme names, so the model reuses themes instead of inventing near-duplicates. Gemini's native JSON mode (`responseMimeType + responseSchema`) constrains the output shape; the response is still run through Zod validation before it ever touches the database. A failed parse retries once, then falls back to a safe "Needs Manual Review" tag rather than losing the item. Classification is stored on the record, never recomputed on page load, and a manual re-classify action is available for corrections.

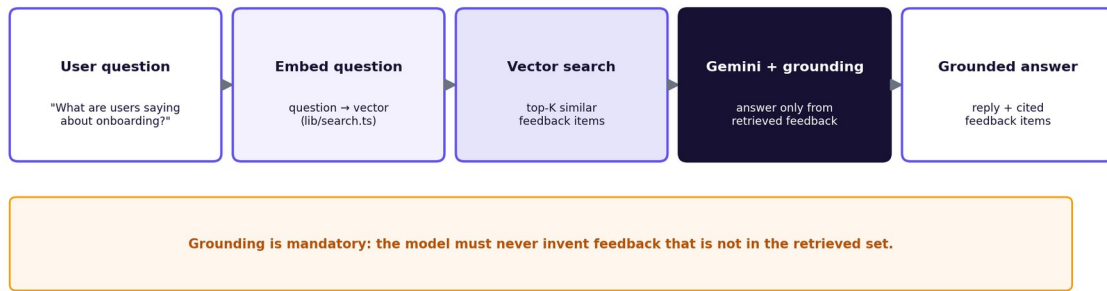
7.2 Theme Clustering & Trends (AI2)

Themes are AI-assigned on ingest. A dedicated trends endpoint computes each theme's volume for the current period versus the immediately preceding period of equal length, and flags a spike when volume is meaningfully up (at least 3 items and a 30%+ increase). Clicking a theme drills into its underlying feedback.

7.3 Ask LOOP — Grounded Q&A (AI3)

Ask LOOP follows a strict retrieve-then-answer pattern: the question is embedded and compared by cosine similarity against the workspace's own feedback embeddings, and only the top-K most relevant items are passed to Gemini as context. The model is explicitly instructed to answer only from those excerpts and to cite which ones it used — it must say so plainly if the data doesn't contain an answer, rather than inventing one.

Ask LOOP — Retrieve, Then Answer (AI3)



7.4 Voice-of-Customer Report (AI4)

Every statistic in a report — item counts, sentiment deltas, top themes, sampled quotes — is pre-computed in code from real data before Gemini is called. Gemini's only job is to write the narrative language around those fixed numbers, so figures can never be hallucinated. Reports are saved, reviewable later, and exportable via the browser's print-to-PDF.

8. Security Model

Non-negotiable security rule

Every database query that touches feedback, themes, reports, or users is filtered by the authenticated user's workspaceId — taken only from the server-side session, never from the request. A user from Company A gets a 404, not a 403, when guessing another tenant's id in the URL, because every lookup is scoped by both id and workspaceId together.

- Passwords hashed with bcrypt (12 rounds) — never stored or logged in plaintext.
- Sessions are signed JWTs delivered as httpOnly cookies via NextAuth.
- Edge middleware redirects any logged-out request on a protected route straight to /login.
- Every API input is validated with Zod before it reaches Prisma — no unchecked client data touches the database.
- GEMINI_API_KEY and DATABASE_URL are server-side only, never bundled to the browser.
- Role permissions are enforced in a single, centralized matrix (lib/rbac.ts) reused by every route — not scattered, ad-hoc checks.

9. Application Walkthrough

The mockups below are illustrative UI renders built to match the shipped component structure and styling (sidebar navigation, card layout, brand colors) — they are not live screenshots. Once deployed, real screenshots from the running app should replace these for the final submission.

9.1 Dashboard

Stat cards plus the three required charts: volume over time, sentiment breakdown, and top themes.

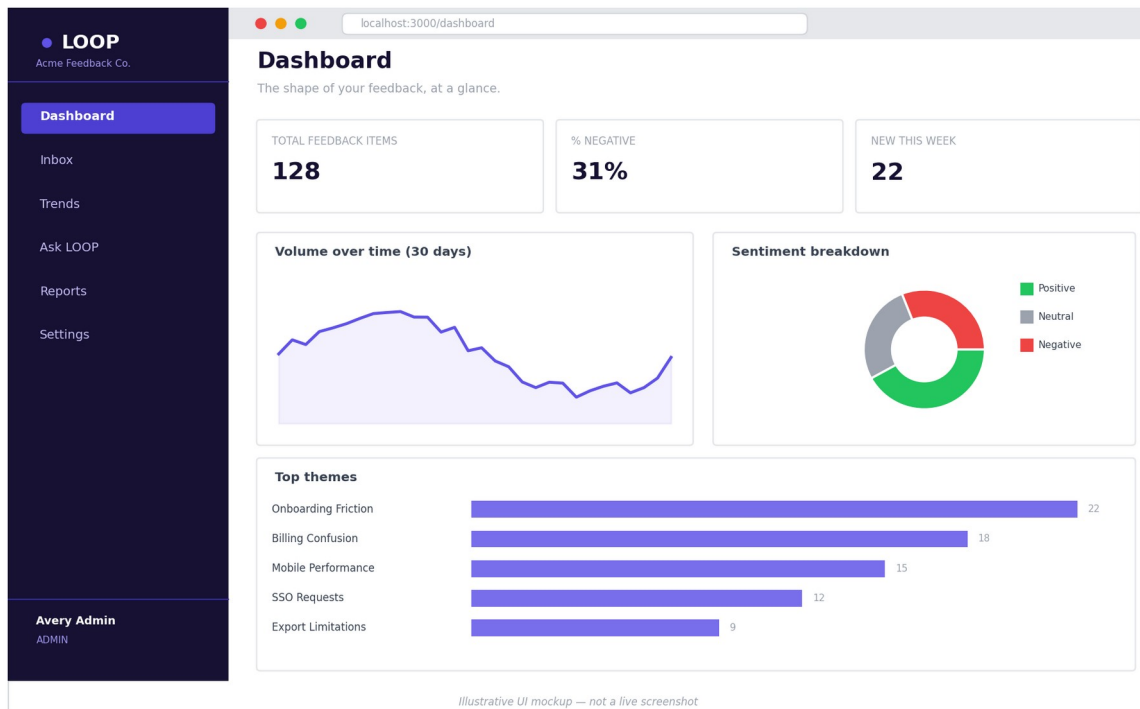


Figure 3. Dashboard — illustrative UI mockup, not a live screenshot.

9.2 Inbox

Filterable, paginated feedback table with inline status changes and theme tags.

The inbox view allows users to search, filter, and manage incoming feedback. It includes a sidebar, a search bar, filter buttons, and a table of feedback items.

Filters:

- Search feedback...
- Channel: All
- Sentiment: All
- Status: All

FEEDBACK	CHANNEL	SENTIMENT	THEMES	STATUS	DATE
Onboarding took forever — couldn't figure out...	Support Ticket	Negative	Onboarding	REVIEWED	Sep 20
The new dashboard is gorgeous and finally fast.	App Store Review	Positive	Dashboard UX	NEW	Sep 21
Prospect wants SSO before they'll sign — 3rd time.	Sales Call Note	Negative	SSO Requests	ACTIONED	Sep 19
It does the job, but mobile needs work.	NPS Survey	Neutral	Mobile Perf.	NEW	Sep 22
Love the new export feature, saved an hour.	Social Mention	Positive	Exports	REVIEWED	Sep 18

User Info: Avery Admin, ADMIN

Illustrative UI mockup — not a live screenshot

Figure 4. Inbox — illustrative UI mockup, not a live screenshot.

9.3 Ask LOOP

A grounded Q&A interface: every answer shows the specific feedback items it was based on.

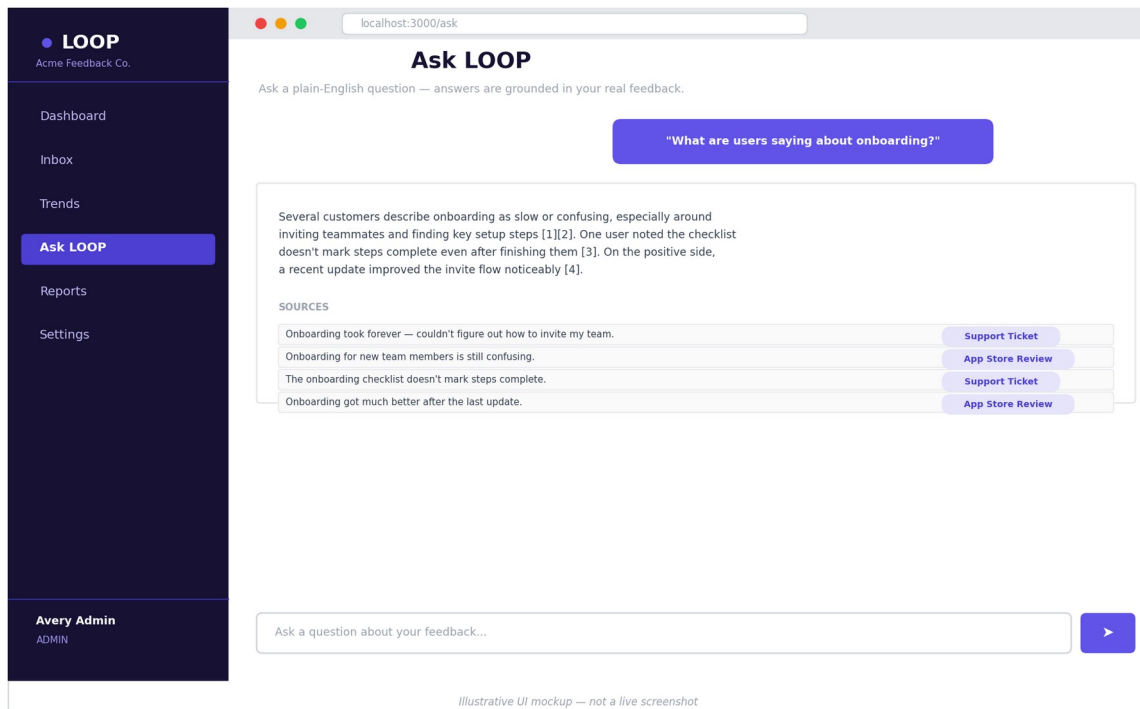


Figure 5. Ask LOOP — illustrative UI mockup, not a live screenshot.

9.4 Reports

Saved Voice-of-Customer reports with pre-computed stats and an AI-written narrative.

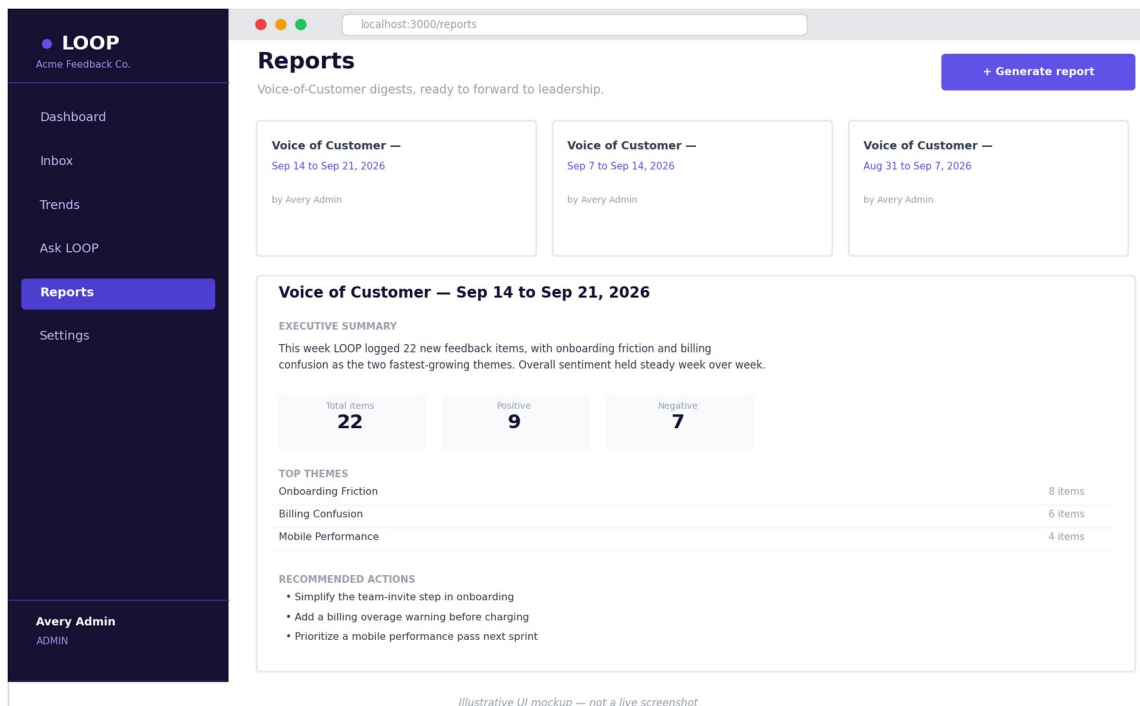


Figure 6. Reports — illustrative UI mockup, not a live screenshot.

10. Deployment

LOOP deploys to Vercel with a hosted PostgreSQL database (Neon or Supabase).

- Code is pushed to a GitHub repository; Vercel builds the Next.js app automatically.
- Environment variables (DATABASE_URL, NEXTAUTH_URL, NEXTAUTH_SECRET, GEMINI_API_KEY) are set in Vercel's project settings.
- prisma migrate deploy applies the schema to the production database.
- npm run seed loads the demo workspace, three role-based users, and ~128 realistic feedback items.
- The live URL is verified end-to-end (login → dashboard → inbox → trends → Ask LOOP → reports) before submission.

11. Known Simplifications

Documented, deliberate trade-offs made to keep the project runnable end-to-end within scope:

- Embeddings use local hashed-TF vectors rather than a hosted embeddings API, avoiding a second vendor API key.
- PDF export for reports uses the browser's native print-to-PDF rather than a server-side PDF library, keeping the dependency footprint small.
- CSV import classification runs synchronously after the batch completes rather than via a background job queue — fine at demo scale; a production version would move this to a queue for very large imports.

12. Conclusion

Project LOOP delivers on the brief's core engineering challenge: a multi-tenant SaaS product with real tenant isolation, server-enforced role-based access control, a validated API layer, and four AI features that require genuine engineering rather than a single cosmetic prompt. The retrieval-grounded Q&A feature in particular demonstrates an understanding of why unconstrained LLM answers are risky in a product context, and how retrieval, structured output, and pre-computed statistics each reduce that risk in a different part of the system.

Building this project required reasoning about the same trade-offs a junior engineer faces at a real product company: what belongs in the API layer versus the UI, how to keep AI output trustworthy without sacrificing usefulness, and how to document simplifications honestly rather than hide them. The result is a submission that reflects full-stack ownership — not a tutorial clone — and a foundation that could reasonably be extended toward the brief's optional stretch goals (saved inbox segments, a suggested-actions queue, sentiment trend alerts, and automated test coverage).

Overall, LOOP meets every required acceptance criterion in the brief's feature specification, is documented thoroughly enough for a stranger to run it locally, and is architected so that its two most deliberate simplifications — local embeddings and browser-based PDF export — can be upgraded independently without touching the rest of the system.